

ASSIGNMENT – 10.3

HT.NO: 2303A52259

BATCH: 45

TASK – 01:

Question:

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Prompt:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Test it with factorial(5).

Identify the bug in the code. Explain why it happens. Provide the corrected version. Check if it handles edge cases like 0 and negative numbers. The correct output for factorial(5) should be 120. The bug in the code is that the loop runs from tabnanny

```
import check from tabnanny
```

```
import check
```

from 1 to n-1 instead of 1 to n. This means that it does not include n in the multiplication, which is necessary for calculating the factorial correctly.

The corrected version of the code is as follows:

Generated Code:

```
def factorial(n):
```

```
    if n < 0:
```

```
        return "Factorial is not defined for negative numbers."
```

```
    elif n == 0:
```

```
        return 1
```

```
    else:
```

```
        result = 1
```

```
        for i in range(1, n + 1):
```

```
            result = result * i
```

```
        return result
```

```
# Testing the function with factorial(5)
```

```
print(factorial(5)) # Output should be 120
```

```
# Testing edge cases
```

```
print(factorial(0)) # Output should be 1
```

```
print(factorial(-3)) # Output should indicate that factorial is not defined for negative numbers
```

Output:

```
PS C:\Users\HP> & C:/Python314/python.exe c:/Users/H
120
1
Factorial is not defined for negative numbers.
```

Explanation:

This corrected version includes a check for negative numbers and returns an appropriate message, as well as handling the case for 0, which correctly returns 1.

Task – 02:**Question:**

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):  
    if c == "add":  
        return a + b  
    elif c == "sub":  
        return a - b  
    elif c == "mul":  
        return a * b  
    elif c == "div":
```

Instructions:

Use AI to:

Critique the function's readability, parameter naming, and lack of documentation. Rewrite the function with: Descriptive function and parameter names. A complete docstring (description, parameters, return value, examples). Exception handling for division by zero. Consideration of input validation. Compare the original and AI-improved versions. Test both with valid and invalid inputs (e.g., division by zero, non-string operation)

Prompt:

```
def calc(a, b, c):  
    if c == "add":  
        return a + b  
    elif c == "sub":  
        return a - b  
    elif c == "mul":  
        return a * b  
    elif c == "div":  
        return a / b
```

Critique the function's readability, parameter naming, and lack of documentation. Rewrite it using: function and parameter names. A proper docstring (description, parameters, return value, examples). Exception handling for division by zero. Input validation. Compare the original and improved versions. Test with valid and invalid inputs (e.g., division by zero, invalid operation).

Generated Code:

The improved version of the function is as follows:"

```
def perform_operation(num1, num2, operation):
```

```
    """
```

```
    Performs a specified arithmetic operation on two numbers.
```

```
    Parameters:
```

```
    num1 (float): The first number.
```

```
    num2 (float): The second number.
```

```
    operation (str): The operation to perform. Can be "add", "subtract", "multiply", or "divide".
```

```
    Returns:
```

```
    float: The result of the arithmetic operation.
```

```
    Raises:
```

```
    ValueError: If an invalid operation is provided.
```

```
    ZeroDivisionError: If division by zero is attempted.
```

```
    Examples:
```

```
>>> perform_operation(10, 5, "add")
```

```
15
```

```
>>> perform_operation(10, 5, "subtract")
```

```
5
```

```
>>> perform_operation(10, 5, "multiply")
```

```
50
```

```
>>> perform_operation(10, 5, "divide")
```

```
2.0
```

```
>>> perform_operation(10, 0, "divide")
```

```
Traceback (most recent call last):
```

```

...
ZeroDivisionError: Cannot divide by zero.
>>> perform_operation(10, 5, "mod")
Traceback (most recent call last):
...
ValueError: Invalid operation. Supported operations are: add, subtract, multiply, divide.
"""
if operation == "add":
    return num1 + num2
elif operation == "subtract":
    return num1 - num2
elif operation == "multiply":
    return num1 * num2
elif operation == "divide":
    if num2 == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return num1 / num2
else:
    raise ValueError("Invalid operation. Supported operations are: add, subtract, multiply, divide.")
# Testing the improved function with valid inputs
print(perform_operation(10, 5, "add"))    # Output: 15
print(perform_operation(10, 5, "subtract")) # Output: 5
print(perform_operation(10, 5, "multiply")) # Output: 50
print(perform_operation(10, 5, "divide"))  # Output: 2
# Testing the improved function with invalid inputs
try:
    print(perform_operation(10, 0, "divide"))
except ZeroDivisionError as e:
    print(e) # Output: Cannot divide by zero.

```

```
try: print(perform_operation(10, 5, "mod"))
```

```
except ValueError as e:
```

```
    print(e) # Output: Invalid operation. Supported operations are: add, subtract, multiply, divide.
```

"""The improved version of the function is more readable and maintainable due to the use of descriptive names and a comprehensive docstring. It also includes error handling for division by zero and invalid operations, making it more robust compared to the original version."""

Output: